四、使用Embedding实现自然语言搜索

有了之前与大模型对接的经历后，接下来，我们将利用AI大模型动手搭建一个RAG智能客服系统。这个系统的核心是要把一些企业额内部的产品手册、常见问题手册等整理成一个知识库，允许用户用自然语言的方式向客服系统询问知识库相关的问题。而智能客服系统可以理解用户的问题，并基于内部的知识库以及大模型的理解能力给出用户想要的答案。这也是AI大模型在很多企业最典型的落地应用的方式。
在动手搭建应用之前，需要带大家了解一个前置的知识： 文本向量化

- 什么是Embedding文本向量化
- 通过向量计算语义相似度
- 向量数据持久化保存
- 链式使用Retriver

1、什么是Embedding文本向量化

AI大模型相比于传统的数据检索，最大的区别在于能够“理解”人类的语言。比如你向ChatGPT问"我是谁"和“我叫什么名字”，ChatGPT都能够把他们正确理解成一个相似的问题。那么计算机是如何理解这些相似的话语之间的意思呢?

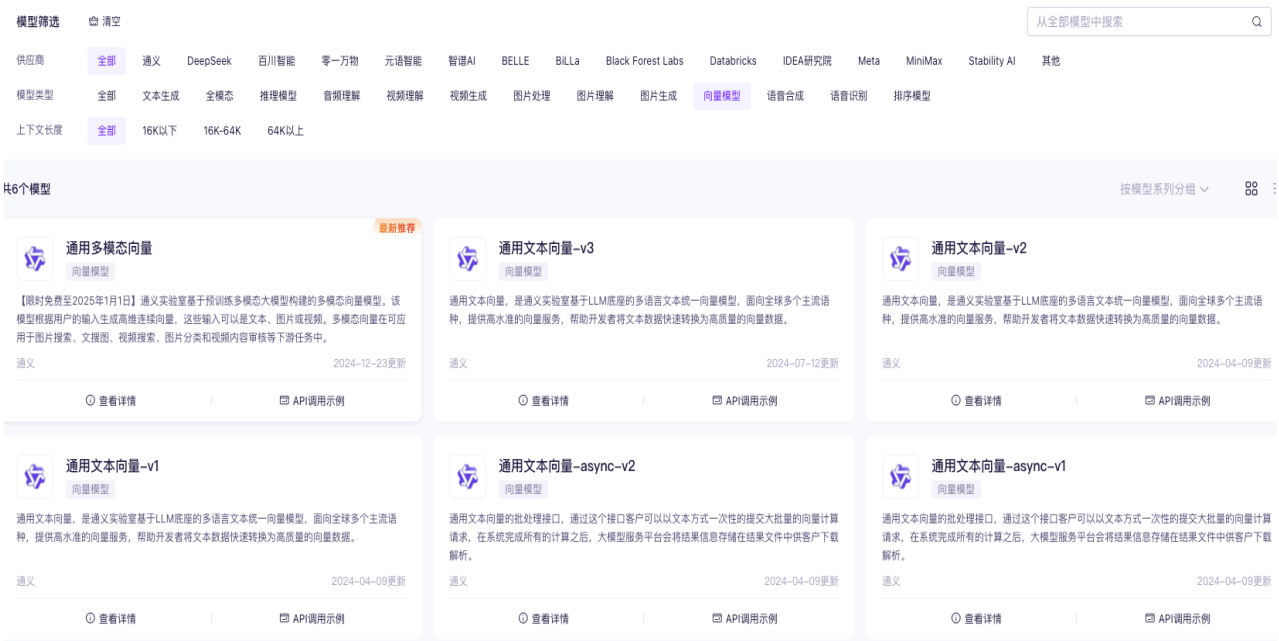
我们需要清楚，计算机并不能“理解”人类的语言，本质上，他只能进行各种各样的数据计算。所以要让计算机“理解”人类的语言，我们只能将语言拆成Token，再将Token数据化，转成一串串的数字，这样才能让计算机通过计算的方式去理解Token之间的相似性，从而进一步理解语言背后的语义信息。

接下来第一个问题就是，我们要如何用数字来表示语言，同时又要保留语言背后的语意呢？这中间经过了一系列的算法改进。目前比较主流的方法是对语言进行一些语法和词法层面的分析，将一个文本转换成多维的向量，然后通过向量之间的计算，来分析文本与文本之间的相似性。

向量代表一个有位置，有方向的变量。一个二维的向量可以理解为平面坐标轴上的一个坐标点(x,y)，他有x轴和y轴两个维度，在计算机中，就可以用一个二维的数组来表示[x,y]。类似的一个多维的数组就对应一个更多维度的向量。

而文本向量化，就是通过机器学习的方式将一个文本转化成一个多维向量，后续可以通过一些数学公式对多个向量进行计算。这既是为了更好的让计算机理解“语言”，同时他也是AI大模型的基础。

实际上，很多AI大模型产品都提供了文本向量化的功能。以国内的阿里云百炼平台为例，就推出了多个不同的向量化模型：



langchain-community社区扩展中也集成了百炼平台向量化的封装，使用起来非常方便。

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
# 安装阿里云百炼的客户端依赖
!pip install -q langchain_community
!pip install -q dashscope
```

```
import os
from config.load_key import load_key

if not os.environ.get("DASHSCOPE_API_KEY"):
  os.environ["DASHSCOPE_API_KEY"] = load_key("BAILIAN_API_KEY")

from langchain_community.embeddings import DashScopeEmbeddings
embedding_model = DashScopeEmbeddings(model="text-embedding-v1")

text = "This is a test query."
query_result = embedding_model.embed_query(text)
print(query_result) # 打印向量结果
print(len(query_result)) # 向量维度
```

> 注意，调用向量化模型，阿里云百炼平台是需要收费的，所以需要提前充点钱。不过目前价格非常便宜，不用担心。

通过Embeddings(langchain中的一个顶层父类)向量化模型，我们就可以把任意文本内容转换成为一个维度固定的向量结构。这样就可以将“语言”的理解问题转换成为数学计算的问题。在langchain中，Embeddings模型是抽象的顶层父类，而具体的Embeddings模型则是继承自Embeddings，并且每个具体的Embeddings模型都对应一个具体的Embeddings模型。
这里需要注意一下的是不同的Embeddings转换出来的向量，维度和结果都是不一样的。当使用一个向量化模型

时，需要额外关注一下向量的维度，因为这会影响到后续对向量的处理。

2、通过向量计算语义相似度

把文本转换成为向量有什么用呢？最核心的作用是可以通过向量之间的计算，来分析文本与文本之间的相似性。计算的方法有很多种，其中用得最多的是向量余弦相似度。 Python语言中提供了一个库sklearn，可以很方便的计算向量之间的余弦相似度。

```
# 安装sklearn依赖
!pip install -q numpy
!pip install -q scikit-learn
```

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# 定义两个文本
text1 = "我喜欢吃苹果"
text2 = "我最爱吃的水果是苹果"
text3 = "今天天气不错"

# 获取文本向量
vector1 = np.array(embedding_model.embed_query(text1)).reshape(1,-1)
vector2 = np.array(embedding_model.embed_query(text2)).reshape(1,-1)
vector3 = np.array(embedding_model.embed_query(text3)).reshape(1,-1)

# 计算余弦相似度
similarity12 = cosine_similarity(vector1, vector2)[0][0]
similarity13 = cosine_similarity(vector1, vector3)[0][0]
print(f"\"{text1}\" 与 \"{text2}\" 相似度: {similarity12:.4f}")
print(f"\"{text1}\" 与 \"{text3}\" 相似度: {similarity13:.4f}")
```

从这个案例看到，text1与text2的语义比较相近，他们计算出来的余弦相似度得分也就比较高。test1与text3的语义比较不相似，所以余弦相似度得分比较低。这种语义相近的计算，其实也是AI大模型理解“语言”的基础。

> 要注意，计算语义相似度的算法其实有很多种，比如有余弦相似度，欧氏距离，曼哈顿距离，等等。不同算法得到的分数含义也是不同的，例如，余弦相似度得分在-1到1之间，得分越高，语义越相似。而欧氏距离得分在0到正无穷之间，得分越低，表示语义越相似。

3、向量数据持久化保存

既然已经能够把文本转换成向量了， 那么自然需要有个工具能够将向量以数据的形式保存起来，并且要能够对向量进行相似度查询。这个工具，就是向量数据库。

LangChain中集成了非常多的向量数据库，具体可以查看官网 [LangChain集成的Vector Stroe]

接下来，我们使用相对比较熟悉的Redis来演示对于向量的一些重要操作。

Redis是一个开源的缓存数据库，拥有集群功能稳定，访问速度快等非常多特性，通常用于存储键值对数据。部署Redis时需要注意，默认开源的Redis社区版本是不支持向量数据存储的，需要额外安装redissearch模块，才能支持向量数据存储。

这里一个大家一种比较简单的部署方式，就是使用docker部署一个带有redissearch模块的redis容器。docker部署指令为

```
docker run -p 6379:6379 redis/redis-stack-server:latest
```

> 注意，目前duckerHub限制了中国大陆的访问。现在docker拉取镜像网上有很多的解决方案，最简单的方法就是科学上网。
> 另外，推荐再安装一个Redis的图形化工具，比如RDM。

有了这个Reids服务后，我们就可以使用LangChain提供的实现类来操作Redis了。

```
# 安装LangChain的Redis扩展
!pip install -q langchain-redis
```

```
redis_url = "redis://localhost:6379"

import redis
redis_client = redis.from_url(redis_url)
print(redis_client.ping()) # 测试连接 返回True表示连接成功
```

```
from langchain_redis import RedisConfig,RedisVectorStore
config = RedisConfig(
    index_name="fruit",
    redis_url=redis_url
)
vector_store = RedisVectorStore(embedding_model,config=config)
vector_store.add_texts(["香蕉很长","苹果很甜","西瓜又大又圆"])
```

```
scored_results = vector_store.similarity_search_with_score("又圆又大的水果是什么",k=3)
for doc, score in scored_results:
    print(f"{doc.page_content} - {score}")
```

> 这个案例中，得分越小表示语义越接近。这可能和底层计算距离的方法有关。

这样，就实现了一个简单的自然语言检索。未来对于用户提出的任何问题，我们都可以在向量数据库中快速检索出和用户问题语义最接近的文本。但是这里需要注意，这里只是检索出我们可能认为最相关的文本，并不能完全保证用户提出的问题，最终的答案就是这个最相关的文本。
基于LangChain框架的良好设计，未来如果想要切换到其他的向量数据库，只需要修改vector_store的实现类即可，业务代码几乎不需要改动。
另外，为了简化这种向量查询，langchain框架中还设计了另外一个类Retriver，来简化这些复杂的检索过程。

```
# 构建检索器，类型为similarity，检索的文档个数为3
retriver = vector_store.as_retriever(search_type="similarity",search_kwargs={"k":3})
retriver.invoke("长长的水果是什么？")
```

4、链式使用Retriver

vector_store.as_retriever方法返回的是一个VectorStoreRetriever类，而这个类也是继承自Runnable类的，所以，Retriver也是可以直接使用LCEL的。

```
from langchain_core.prompts import ChatPromptTemplate
# 创建提示模板
prompt = ChatPromptTemplate.from_messages([
    ("human", "{question}"),
])
# 格式转换函数，prompt.invoke方法返回PromptValue，而retriver.invoke需要传入的参数为str。中间做个格式转换
def format_prompt_value(prompt_value):
    return prompt_value.to_string()
# 链式连接检索器和提示模板
chain = prompt | format_prompt_value |retriver
# 调用链并传入用户的问题
documents = chain.invoke({"question":"又长又甜的水果是什么？"})
for document in documents:
    print(document.page_content)
```

总结

这一章节中，我们开始接触到了计算机理解自然语言的方式-向量。通过向量相似度的计算，我们可以让计算机能够在一定程度上理解自然语言。这也正是AI大模型区别于传统搜索引擎的根本之处。
围绕向量，Langchain框架提供了一系列完整的设计，来支持各种对于向量的操作。有了这些设计后，我们就可以很方便的集成各种工具来完成围绕向量的各种操作。这些顶层父类包括：

- Embeddings： 代表文本向量化模型
- VectorStore: 代表持久化保存向量的向量数据库
- Retriever: 代表向量数据库的检索器
  逐步了解并总结langchain的这些顶层设计，不光能够帮助我们快速构建本地应用，更重要的是，通过这些顶层设计，langchain帮我们沉淀了应该如何更好的使用AI大模型的应用经验。